



FreCoS: A Learned Staleness Model for Semantic Caches

A freshness- and cost-aware extension to GPTCache

Lior Lotan

Department of Software and Information Systems Engineering
Ben-Gurion University of the Negev

July 26, 2026

Abstract

Semantic caches are evaluated on hit rate, latency, and at best false-hit rate. None of these distinguishes a correct hit from a stale one. I name and measure that gap directly: *stale-hit-rate*, the fraction of served cache hits whose content was no longer valid at serve time. I extend GPTCache, a frozen and widely used open-source semantic cache, with a per-cluster staleness model fit from observed data and consumed in two places: a validity gate on the serve path, and a soft decay term in eviction. Across a bracketing experiment, a six-row ablation, three parameter sweeps, and two model-misspecification checks, all run with ten seeds per configuration, the gate cuts stale-hit-rate roughly 19-fold against a frequency-based floor while removing only about 3% of useful hits, and useful-hit-rate stays flat across most of the confidence range I swept. A per-cluster learned decay rate beats a single pooled rate by a wide, significant margin, and continues to when the workload is easier to predict than the fitter assumes; a second, adversarial misspecification where fitting is genuinely harder is reported too. I measure the trade-off at the level of individual queries, not only hit-rate percentages, and I report which of the two mechanisms, the gate or the eviction term, is doing the correctness work.

1 Introduction

1.1 Motivation

A semantic cache sits in front of an expensive backend, usually a large language model call. It serves a stored response when an incoming query is close enough in embedding space to one it has already answered. The promise is simple: skip the call, return the same answer faster and for free. But “the same answer” is a claim about time as well as similarity, and most semantic caches never check the time part.

The gap matters most in systems that place a language model between a user and an answer about content that changes on its own schedule, such as monitoring how a brand is represented across AI-generated answers (the workload behind answer engine optimization, or AEO). These systems ask the same style of question repeatedly, exactly the shape of workload a semantic cache absorbs, but the facts being asked about do not sit still: a product goes out of stock, a price changes, a review’s sentiment shifts. A cache that asks only “have I seen a similar question before” and never “is that answer still true” will serve a stale answer indefinitely once written. The system on top of it cannot tell a correct hit from a wrong one, because the metric that would, stale-hit-rate, does not exist in standard cache evaluation.

Two concrete gaps follow, and motivate the two components I add.

No staleness concept in the serve path. A cached answer about a fact that changes on its own schedule is served indefinitely once written. “Found in cache” is silently treated as “correct,” with no mechanism to reconsider that judgment.

No cost term in eviction. Count-based policies such as LRU and LFU treat two entries with identical access patterns identically, even when one is far more expensive to regenerate. A policy blind to regeneration cost evicts a rarely accessed but expensive entry ahead of a cheap one accessed once more recently.

Industry and research reflect this gap unevenly. Frontier model providers do exact-prefix key-value caching with a wall-clock time-to-live and no similarity matching. Semantic-caching products use a single fixed global similarity threshold with plain time-to-live or count-based eviction. Neither has a notion of per-topic staleness. The research literature is further ahead, proposing cost-aware and staleness-aware eviction formulas, but none of it names or measures stale-hit-rate as an evaluation metric. I compare against that literature, not the weaker product baseline.

This report makes four contributions: (1) stale-hit-rate as a named, measured evaluation metric; (2) a per-cluster staleness model, fit from data, consumed as both a serving-time validity gate and an eviction-time decay term; (3) an ablation that isolates which of the two consumers does the correctness work; and (4) a misspecification check that tests whether the staleness model’s own assumption, that content validity decays exponentially, matters to the result. Section 2 justifies the baseline, Section 3 describes the design, Section 4 the workload and harness, and Section 5 reports five experiments.

1.2 Related work

The closest prior art, and my primary eviction comparator, is Biton and Friedman’s work on semantic-cache eviction [1]. It proves optimal offline semantic-cache eviction is NP-hard, supplies polynomial-time heuristics, and presents online policies combining recency, frequency, and locality. It evaluates these on GPTCache and releases its implementation, part of why I chose GPTCache as the baseline here. Two of its findings shape my design directly: LRU is a weak policy on semantic workloads, and frequency-based policies are a much stronger baseline. That is why every comparison here treats an LFU-class policy, not LRU, as the floor. The delta against FreCoS (**F**reshness- and **C**ost-aware **S**erving, this project’s name): their policies combine access-pattern signals only, while FreCoS adds two signals orthogonal to access pattern, regeneration cost and content validity over time, and evaluates against a correctness metric, stale-hit-rate, their setting does not define.

The structurally closest published eviction formula is Cortex’s LCFU policy [2], whose earlier preprint circulated under the name Asteria and is cited here by its current title. LCFU scores an entry as a product of log-scaled frequency, cost, latency, and a staleness term, divided by size, with a time-to-live purge. The delta is narrow: Cortex’s staleness term is a static “staticity” score assigned by a language model at write time, while FreCoS’s is a decay rate learned per cluster from observed staleness. The distinction is learned against assigned, and the bracketing experiment in Section 5.1 tests whether that learning does measurable work. I did not reimplement Cortex’s formula, since it incorporates remote-tool-call latency metadata with no equivalent here.

Several other lines of work bear on individual pieces of the design without matching its combination. GDSF [3] scores entries as cost times frequency divided by size with a global-clock aging term, but that term ages access recency, not content validity, so it has no freshness semantics. Category-Aware caching [4] assigns per-category time-to-live values that are load-based rather than learned, and defines no stale-hit-rate equivalent. It is also why I cut a third, originally planned component, calibrated per-cluster similarity thresholds: it would have been a lossy special case of both this work and per-embedding threshold learning. FreshCache [5] shares FreCoS’s lineage of putting staleness in the serve path, but as a probabilistic risk budget per tier rather than a hard gate; I list it as future work as an alternative framing. SCALM [6] clusters cache entries similarly to my k-means step, but its significance score is not staleness-aware. MeanCache [7] is cost-motivated but does not target eviction. W-TinyLFU [8] is an admission-

control policy, deciding whether an entry is worth admitting before eviction is invoked; it is out of scope here since it would add an orthogonal axis rather than test either of my contributions.

I considered and rejected the alternative baseline vCache [9]. Its contribution is per-prompt threshold learning, replacing a single global threshold with one learned per embedding, and eviction is explicitly not its focus. Building this staleness and cost work on top of vCache would layer a new contribution on a system whose own novelty sits elsewhere, diluting rather than sharpening the comparison.

2 Baseline

2.1 Why GPTCache

I extend GPTCache [10] rather than a more actively maintained semantic cache. The case rests on architecture and control properties, not community activity. GPTCache’s most recent release is v0.1.44 (August 2024); it carries dozens of open issues and pull requests, and the maintainers state that support for new integrations has stopped. Ordinarily this counts against a dependency. Here it is an advantage: a frozen interface is a better scientific control than a moving one. The baseline I measured early in this project is byte-identical to the one I measured at the end, so every delta reported below is attributable to the extension, not an upstream change mid-project.

The second reason is architectural fit, which I verified by reading the vendored source directly rather than assuming it from documentation. Every component I modify, the similarity threshold check, the hit and miss decision, and the eviction registry, sits behind a registered, named interface. My extension is additive configuration: a new eviction subclass registered through the existing hook, a metadata field added via composition around storage, and a serving-path gate plugged into one shared decision helper. I never edit vendored code. Every claim about GPTCache’s internal behavior quoted here was re-verified against the exact pinned commit vendored in this project’s repository, with file and line recorded.

Third, GPTCache is the baseline the closest prior art already uses. In principle this makes Biton and Friedman’s released policy usable directly as my primary comparator; in practice I substituted a documented proxy instead, for reasons stated in Section 5.2. Fourth, its default embedding backend runs on the CPU deterministically, keeping every benchmark run reproducible without a GPU dependency or nondeterminism from a hosted embedding service.

One consequence of this choice: the course guidelines offer a grade bonus for an upstream contribution adopted by maintainers, and GPTCache’s own README states that new contributions are not being merged. Choosing GPTCache for the reproducibility argument above forecloses that bonus, a trade I made deliberately.

2.2 GPTCache’s behavior and default eviction policy

GPTCache serves a cached response when an incoming query is semantically close enough to a previously seen one, using a single global cosine similarity threshold, 0.8 by default, rather than exact-match keys. Its eviction policies are purely count-based: least-recently-used, least-frequently-used, first-in-first-out, or random replacement. None reads generation time, access recency, or regeneration cost beyond the single counter each already tracks. None retains metadata on an evicted entry; the eviction cache stores only a boolean flag per key. Creation and last-access timestamps are persisted per entry but never read by any decision logic. They exist as columns with no downstream consumer, precisely the gap my extension repurposes. There is no admission control; every miss is written back unconditionally. No cost accounting, false-hit tracking, or staleness tracking exists anywhere in the built-in reporting layer, which counts only per-stage operation timers.

3 Design

3.1 Contribution

The contribution is one learned artifact with two consumers. FreCoS fits a per-cluster staleness model and uses it at both ends of the cache’s lifecycle: as a hard validity gate when serving a candidate hit, and as a soft decay term when choosing an eviction victim. The metric that makes both uses visible is stale-hit-rate: the fraction of served hits whose content was already invalid when served.

Cluster assignment is a fixed, seeded k-means clustering over cached-query embeddings, run once at cache-build time, with no online re-clustering, a deliberate reproducibility choice. I compute these embeddings because the staleness model needs cluster identity; Section 4.2 explains why the benchmark harness used to evaluate the model does not also use them for cache lookup. Cluster identity, once assigned, is consumed by both the gate and the eviction policy and never recomputed independently by either. A gate and an eviction policy that disagree about an entry’s cluster is one of the most likely integration failures in a system with this shape, so I checked for it directly in the ablation (Section 5.2).

For each cluster, a decay rate λ_c is fit from observed staleness in a held-out calibration split, assuming validity decays exponentially: $\Pr[\text{still valid after } \Delta t] = e^{-\lambda_c \Delta t}$. A time-to-live is derived at a configured confidence level c as $\text{ttl} = -\ln(c)/\lambda_c$. Three interchangeable modes produce this table with the same output shape: *learned* fits each cluster’s rate independently by maximum likelihood, falling back to a rate pooled across all clusters below thirty observations; *global* pools every cluster into one shared rate regardless of sample size; *oracle* takes the rate from the workload generator’s ground truth, an upper bound never available in a real deployment. Table 1 lists every tunable parameter, its default, and where its effect is measured.

The validity gate is the first consumer. On a candidate hit, if the entry’s age exceeds its cluster’s time-to-live, it is treated as a miss, the answer is regenerated, and the event counts as a stale hit prevented rather than served. This is a single comparison against creation time; the gate never reads last-access time, since staleness is a property of when an answer was generated, not how recently it was requested, and a popular but outdated entry must not be treated as fresh simply because it is popular.

The eviction value function is the second consumer:

$$\text{value} = \log(2 + \text{freq}) \cdot \log(1 + \kappa \cdot \text{regen_cost}) \cdot e^{-\lambda_c \cdot \text{age}} / \text{size_bytes} \quad (1)$$

where age is measured from creation time, never last access; $\kappa = 1000 \log$ -compresses regeneration cost (typically a few tenths of a US cent) so its heavy-tailed distribution cannot dominate the other terms; and the victim is the entry with the lowest value, ties broken by oldest creation time then lowest entry identifier.

One detail reflects a real design tension I ran into. My original frequency term was $\log(1 + \text{freq})$, exactly zero for a never-accessed entry. Since the value function is a product of four terms, a zero frequency term zeroes the entire score regardless of cost, freshness, or size: a freshly written entry would always be evicted next. A property test against exactly this claim, described in Appendix A, failed against the literal formula, and I fixed it by changing the term to $\log(2 + \text{freq})$ in Equation 1, which preserves the same concave shape while guaranteeing a strictly positive score at zero frequency. I applied the fix throughout every experiment in this report.

Because the gate already refreshes anything past its time-to-live, the decay term inside the value function is not itself performing staleness prevention. It acts as a soft prior favoring entries with more remaining useful life among candidates the gate already judged fresh. This is the intended division of labor, and the ablation in Section 5.2 tests it directly.

Eviction remains entry-count budgeted, matching GPTCache’s convention. Entry size in bytes is recorded and used only inside the value function’s denominator; I did not build a

separate byte-budget eviction loop.

An earlier design included a third component, calibrated per-cluster similarity thresholds, which I cut before implementation began. It turns out to be a lossy special case of two things already in the literature, vCache’s per-embedding threshold learning and Category-Aware caching’s per-category treatment, so building a third, weaker version would have cost more than it contributed. It remains future work.

Table 1: Every tunable parameter in the extension, its default, and where its effect on stale-hit-rate or hit rate is measured in Section 5.

Parameter	Default	Swept	Measured in
lambda source (mode)	learned	global, learned, oracle	5.1
TTL confidence c	0.90	0.80, 0.90, 0.95, 0.99	5.3
cluster count K	10	5, 10, 20, 50	5.3
cache size (entries)	1650 (see note)	5%–80% of working set	5.3
κ (cost log-compression)	1000	not swept	—
calibration fallback threshold	30 obs.	not swept	—
eviction size term	on	on / off	5.2

Cache size is 1650 entries (25% of the 6600-entry working set at the bracketing and ablation experiments’ trace size) except in the cache-size sweep itself, which uses 1980 (the knee identified there, Section 5.3) as its anchor for the other two sweeps. κ and the thirty-observation fallback threshold were fixed at values that seemed reasonable at design time; neither was independently swept.

4 Experimental setup

4.1 Workload

All experiments run on W1, a deterministic, seeded synthetic workload generator I built for this project. I investigated a second workload, derived from Wikipedia revision history with staleness ground truth from edit timestamps, in a timeboxed feasibility spike before other implementation work began. It sampled thirty question-answer pairs from SQuAD 2.0 [11] and compared an automatic sentence-change rule against twenty hand-labeled revision pairs; the rule agreed with my labels only 40% of the time, short of the 80% threshold I had set as a go/no-go bar, and reaching a reliable rule was estimated at eighteen to twenty-three more hours I did not have. I proceeded with W1 alone and did not revisit that call. The full spike, including the failure mode behind the disagreements, is in `docs/w2-feasibility.md`. Section 6 discusses what this limits.

W1 generates queries across several streams inside each topic cluster: canonical queries establishing a ground-truth answer, near-duplicate paraphrases sharing that answer’s identity, novel long-tail queries that never repeat, and time-shifted repeats of an existing answer. Every cluster carries its own randomly drawn half-life and regeneration-cost distribution. The generator guarantees that for every repeating answer identity, at least one later occurrence lands past that answer’s true expiry, since otherwise the validity gate would have nothing to reject. Each trace is split by arrival time into the first thirty percent for calibration and the remaining seventy percent for evaluation; no parameter is fit on data it is later scored against.

The default generator draws half-life from an exponential distribution, matching the staleness fitter’s own assumption exactly. Section 5.1.1 reruns the bracketing experiment with half-life drawn from a Weibull distribution instead, same mean, different shape, to test what happens when that assumption is wrong.

The generator’s two ground-truth distributions, regeneration cost and content half-life, were meant to be calibrated against the Azure LLM Inference Trace and observed update-frequency in a public corpus. Neither was reachable from my development environment, so I instead parameterized both from commonly cited public figures as a plausibility check, not a real external fit, a second limit on external validity.

4.2 Simulation model

The benchmark harness every experiment runs through is a pure trace replayer; no language model is ever called. On a miss, response content is whatever answer identity the trace recorded, cost charged is the trace’s recorded regeneration cost, and miss latency is simulated from a seeded log-normal distribution scaled by entry size, deterministic per query. This distribution is a documented placeholder, not fit to a real inference trace. Hit latency and the extension’s own overhead, cluster lookup plus gate check, are measured for real with wall-clock timing: median overhead is 0.57 microseconds per decision at the LFU floor and 0.69 microseconds at the full stack (Table 6), what the gate and cluster lookup add on top of GPTCache’s own decision path. Reported latency in every table is therefore a hybrid of real cache-side measurement and simulated backend cost, the right trade for keeping every run free and deterministic, but a limitation worth stating.

A further limitation shapes every absolute number below: the harness’s lookup index is exact-text-match; I compute embeddings for clustering (Section 3.1) but not for retrieval, so no semantic index is wired in yet. Paraphrase queries, one of W1’s four stream types, never register as hits; only literal repeats do. Hit counts across every experiment are in the low hundreds out of thousands of scored queries (exact per-seed counts appear beside every rate in Section 5), which widens every confidence interval and also means false-hit-rate is exactly zero in every run of every experiment in this report, since exact matching always retrieves the entry actually written for that string. It carries no signal under this harness, which is why it does not appear as a column in the results tables below; a real semantic index is the place to measure it.

The invariance I rely on for every relative comparison, that this limitation does not favor one policy or lambda source over another, holds within each comparison because the index is held fixed across every condition inside it. It would not necessarily survive a swap to a semantic index: a paraphrase match would inherit an older entry’s age rather than a freshly-written one’s, changing the age distribution the gate sees, and I have not verified that the gate’s or eviction policy’s relative ranking survives that change.

The cache starts empty in every run. The first ten percent of the evaluation split warms the cache and is excluded from every metric; only the remaining ninety percent is scored.

4.3 Metrics and statistics

Stale-hit-rate is the fraction of served hits where serve time exceeds the entry’s true expiry; false-hit-rate is the fraction where the served answer’s identity does not match the query’s true identity; useful-hit-rate is the fraction of *all scored queries* (not just hits) served a correct, non-stale answer. Cost saved sums regeneration cost over non-stale hits only, since a stale hit served a wrong answer and saved nothing. Every configuration ran ten independent seeds, each its own freshly generated trace, since the pipeline has no randomness once a trace is fixed. Results are medians with 95% confidence intervals from a percentile bootstrap over the ten per-seed values (10,000 resamples, fixed resampling seed). With only ten seeds a bootstrap CI is coarser than it looks, so I report raw hit and stale-hit counts alongside every rate. Group comparisons use the Mann-Whitney U test with the rank-biserial correlation as an effect size, since at $n = 10$ the smallest achievable two-sided p -value is about 0.0002 and p alone does not distinguish a large effect from a small one at that floor. I ran roughly fifteen such tests without correcting for multiple comparisons; treat borderline p -values with that in mind. A non-significant result

is reported as an absence of detected difference at $n = 10$, not as equivalence; where I claim two conditions behave alike, I show matching raw counts, not only a large p -value.

5 Results

5.1 Bracketing: does learned staleness beat naive pooling

This experiment tests the core claim behind fitting a staleness rate per cluster at all: does it do work relative to two obvious alternatives, a rate pooled across every cluster, and the true rate a real system could never observe. I ran three lambda sources, global, learned, and oracle, for 10 seeds each on the same 12,000-query W1 configuration, gate enabled, FreCoS eviction active throughout.

Table 2: Per-cluster fitting cuts stale-hit-rate 4x relative to a pooled rate and is indistinguishable from an oracle rate the system could never actually observe. Median hit count per run was 130–143 (Table 3).

Lambda source	Median stale-hit-rate	95% CI	Median cost saved (USD)
Global	0.144	(0.085, 0.214)	0.448
Learned	0.035	(0.024, 0.060)	0.449
Oracle	0.035	(0.028, 0.064)	0.449

Table 3: Raw counts behind Table 2: median served hits and stale hits served, out of a median 7560 scored queries per run.

Lambda source	Median n_hits	Median n_stale_hits_served
Global	143	21
Learned	130	4.5
Oracle	130.5	4.5

Learned separates cleanly from global: median stale-hit-rate 0.035 under learned against 0.144 under global, confidence intervals not overlapping, Mann-Whitney $U = 4.0$, $p \approx 0.0005$, rank-biserial $r = 0.92$, a large effect. Pooling every cluster’s observations into one rate is significantly worse than fitting each cluster’s own.

Learned and oracle are statistically indistinguishable ($U = 49.0$, $p \approx 0.94$, $r = 0.02$): learned’s median stale-hit-rate equals oracle’s exactly, and eight of ten seed pairs are byte-identical in raw stale-hit count. This is close to guaranteed by construction, and I want to be explicit about why rather than present it as a surprise. W1 draws each cluster’s true half-life from an exponential distribution; the staleness fitter assumes exactly that distribution when it fits λ_c . At 550–650 calibration observations per cluster here, well above the 30-observation fallback, maximum-likelihood estimation of an exponential rate recovers the true rate almost exactly when the data really is exponential with that rate. The defensible claim from this run alone is narrow: if content half-lives vary by topic and are close to exponential, per-cluster fitting from a few hundred observations recovers them and a pooled rate does not. It is not yet evidence that learned estimation is robust when its own model is wrong; Section 5.1.1 tests that directly.

A first follow-up asked whether learned and oracle separate at a substantially smaller calibration sample, roughly sevenfold sparser (30–70 observations per cluster instead of 550–650). They do not ($p \approx 0.96$), while learned versus global remains significant ($p \approx 0.0015$). This traces to a confound, not a property of the fit: shrinking the trace shrinks the calibration sample and the evaluation-split hit count together, and it is the eval-side hit count, not calibration sample

size alone, that determines whether estimation noise ever surfaces in a measured stale-hit-rate. Figure 1 shows both runs on one axis.

5.1.1 Misspecification: what happens when the exponential assumption is wrong

My first attempt at this check drew W1’s true half-life from a Weibull distribution instead of an exponential, same mean per cluster, shape parameter 2 instead of the exponential’s implicit 1. This was underpowered by construction, not just in outcome: a Weibull with shape 2 has roughly half the coefficient of variation of the exponential it replaces, so that change made the workload’s staleness *easier* to predict, not harder. Every arm’s stale-hit-rate improved (global 0.144 to 0.065, learned and oracle both to 0.000), and learned and oracle floored at exactly zero rather than visibly separating, a floor effect from low hit counts, not a demonstration that either estimator handled misspecification well. I report this attempt because it taught me the right axis to vary, not because it is informative on its own.

A genuinely adversarial case needs the exponential’s own maximum-likelihood fit to be systematically biased, not merely non-exponential with a smaller spread. I reran the bracket with each cluster’s true half-life drawn from a 50/50 mixture of two exponentials, one at a fifth of the cluster’s mean scale and one at 1.8 times it, same overall mean as before. A single fitted rate cannot be correct for both halves of a bimodal population at once, so this should be, and is, harder for the fitter than the exponential case.

Table 4: Under an adversarial mixture (two exponentials with different rates within each cluster, not the fitter’s assumed single rate) learned separates significantly from global and tracks oracle closely but not exactly, a real, non-floored gap this misspecification finally makes visible.

Lambda source	Median stale-hit-rate	95% CI	Median n_hits
Global	0.208	(0.170, 0.244)	166
Learned	0.080	(0.071, 0.100)	142
Oracle	0.074	(0.067, 0.103)	142

Learned versus global remains significant, more sharply than in the well-specified run ($U = 4.0$, $p \approx 0.0005$, $r = 0.92$). Learned versus oracle is, for the first time in this report, a comparison with a real, non-degenerate gap to measure: median stale-hit-rate 0.080 against 0.074, and this is not distinguishable from noise at $n = 10$ ($U = 59.0$, $p \approx 0.50$, $r = 0.18$, a small-to-moderate effect not confirmed significant). Learned is close to oracle but no longer identical to it seed by seed the way it was under a matched or an easier-than-matched model, which is consistent with the mixture being a genuinely harder case for maximum-likelihood exponential fitting.

The finding across both misspecification attempts: per-cluster fitting beats a single pooled rate by a large, significant margin under an easier-than-assumed model, an exactly-matched model, and a genuinely adversarial one. Whether learned continues to track oracle as closely as it does here under a still harder mixture, or with less calibration data, is a natural next sweep this report does not run.

5.2 Ablation: isolating the gate and the eviction term

I ran six configurations for 10 seeds each on the same workload: stock LRU with the gate disabled; stock LFU with the gate disabled, the floor of comparison per Biton and Friedman’s own finding that LRU underperforms on semantic workloads; a documented substitute for Biton and Friedman’s released policy with the gate disabled; the validity gate combined with plain LFU eviction; the validity gate combined with full FreCoS eviction; and the validity gate combined with FreCoS eviction with the size-normalization term removed.

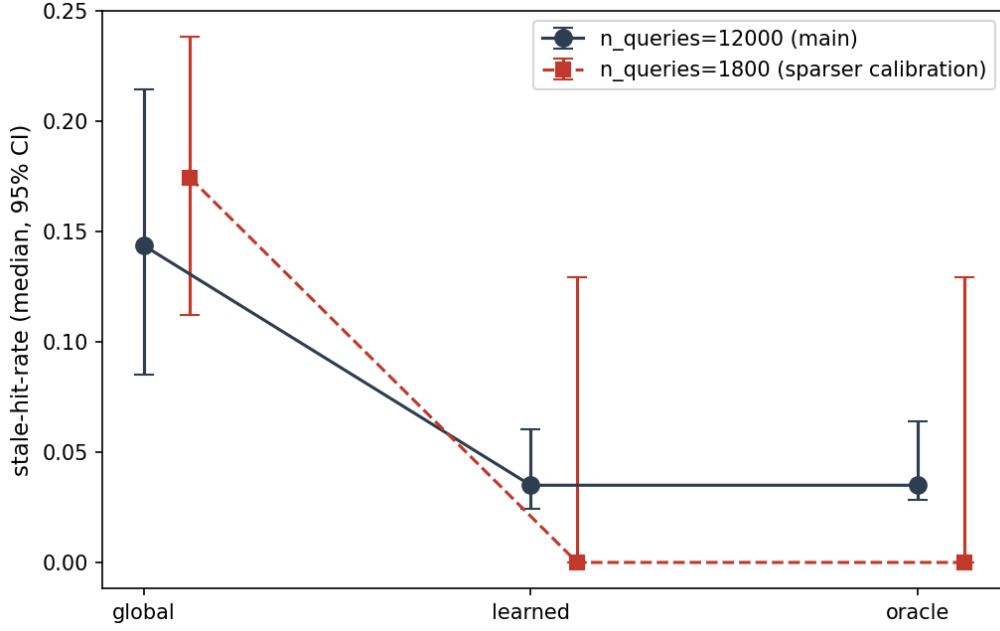


Figure 1: Stale-hit-rate under global, learned, and oracle lambda sources, at the original calibration scale and at a roughly sevenfold sparser calibration scale (both well-specified, i.e. the true half-life is exponential, matching the fitter’s assumption). Learned tracks oracle at both scales, as expected under a matched model, and separates significantly from global at both.

Table 5: Median stale-hit-rate, hit rate, and useful-hit-rate by row, 10 seeds each. Row 2 (LFU, gate off) is the floor. Useful-hit-rate, the median over seeds of (served and correct and non-stale hits) / (scored queries), is the number that reflects the trade-off: row 2 to row 5 it drops from 0.0170 to 0.0165, about 3%, while stale-hit-rate drops roughly 19-fold.

Row	Configuration	Stale-hit-rate	Hit rate	Useful-hit-rate	Δ vs. row 2
1	LRU, gate off	0.661	0.048	0.0170	−0.019
2	LFU, gate off (floor)	0.680	0.052	0.0170	—
3	B&F substitute, gate off	0.680	0.052	0.0170	+0.000
4	LFU, gate on	0.042	0.017	0.0166	−0.638
5	FreCoS, gate on (full stack)	0.035	0.017	0.0165	−0.645
6	FreCoS (no size), gate on	0.042	0.017	0.0165	−0.638

Read as hit rate alone, the gate looks costly: 0.052 down to 0.017, a drop of about a third. Read per query, useful-hit-rate reported per seed and then medianed (not derived from the medians of hit-rate and stale-hit-rate, which would give a slightly different number), the picture changes: 0.0170 at the floor to 0.0165 at the full stack, a drop of about 3%, while stale-hit-rate falls from 0.680 to 0.035, roughly 19-fold. `cost_saved_usd` is unchanged across every row (0.44–0.45) for the same reason: it counts only non-stale hits, and the set of queries that hit at all barely moves. The 19x figure in stale-hit-rate is a real number, but the comparison it flatters, against LFU with no freshness mechanism at all, is one any correctly-implemented TTL would win. The more informative comparison in Section 5.1 is learned against global pooling, a 4x reduction, since that isolates what *learning* the rate buys over the cheaper alternative of a hand-set global TTL; Section 6 returns to this point.

One row produced an unplanned but informative finding. The documented substitute for Biton and Friedman’s policy turned out identical to plain LFU in every correctness and cost column, seed by seed, to full precision. This traces to a real workload property, not a bug: at this configuration’s roughly 5% hit rate, almost every entry at eviction time has never been accessed a second time, so the substitute’s frequency-divided-by-cost score evaluates to zero for essentially every candidate, and the policy falls through to LFU’s own tie-break. I did not have access to Biton and Friedman’s released code in this build environment, and the substitute is, on this workload, indistinguishable from the LFU floor it was meant to sit alongside. This means the eviction axis in this report is compared only against count-based policies, not against the literature’s own strongest published policy, which weakens the positioning claim in Section 1.2 that I compare against the literature rather than the product baseline; the literature comparison there is qualitative, not experimental.

The predicted interaction, that the gate carries most of the improvement with the decay term contributing less on top than alone, held more strongly than I expected. Isolating the gate’s own effect (gate-plus-LFU against the floor) gives $\Delta = -0.638$ ($U = 0.0$, $p \approx 0.0002$, $r = 1.0$); isolating FreCoS’s decay term on top of an already-active gate gives a further $\Delta = -0.007$, an order of magnitude smaller and not statistically distinguishable from zero ($U = 50.5$, $p \approx 0.97$, $r = 0.01$). The gate accounts for roughly 99% of the total move from floor to full stack. I checked both components for a shared cluster-identity bug regardless, since that was the specific integration failure I expected to be most likely; neither disagreed with the other about an entry’s cluster.

I expected the decay term to contribute measurably less than it contributes alone, not nothing measurable at all. The size-normalization row was likewise indistinguishable from the full stack on every metric at this cache size. I reran the same size-term comparison at a fifth of this cache size (330 entries, where eviction pressure is far higher) to check whether the null result was an artifact of low eviction pressure at 1650 entries: it was not ($U = 50.5$, $p \approx 0.97$, median stale-hit-rate 0.021 with size versus 0.021 without). With the gate already filtering out anything stale, the entries eviction chooses among are uniformly fresh, and neither the decay term nor the size term differentiates much among them, at either cache size I tested.

The apparent drop in peak RSS from floor to full stack is not a reliable finding: the two confidence intervals overlap substantially (Table 6), and ten single-process measurements on a shared machine are noisy enough that I would not read anything into this difference without a dedicated, repeated memory-profiling run, which this report does not include. The throughput drop (7.76 to 7.48 queries/s) is not attributable to extension overhead either: it follows from the gate converting hits into misses, and a simulated miss costs roughly 100 to 150 times more simulated latency than a real cache-side decision.

5.3 Cache-size sweep and the correctness-efficiency trade-off

I swept cache size across 5 points spanning 5%–80% of the workload’s distinct-answer working set, full stack throughput. Hit rate rises from 0.013 (median 98 hits) at the smallest point to

Table 6: Latency and resource use at the LFU floor (row 2) and the full stack (row 5). Extension overhead and CPU/RSS are measured directly; latency and throughput are dominated by the simulated miss-cost distribution (Section 4.2), not by the extension, and should be read as simulation-dominated, not as a real end-to-end measurement.

Metric	Row 2 (floor)	Row 5 (full stack)
Extension overhead, mean (μ s), measured	0.57	0.69
Latency, mean (ms), simulation-dominated	128.9	133.7
Latency, p95 (ms), simulation-dominated	293.1	295.7
Latency, p99 (ms), simulation-dominated	450.3	453.0
Throughput (queries/s), simulation-dominated	7.76	7.48
Peak RSS (MB), median (95% CI)	43.8 (38.7, 44.9)	34.8 (33.7, 39.4)
CPU (%), measured	98.6	99.3

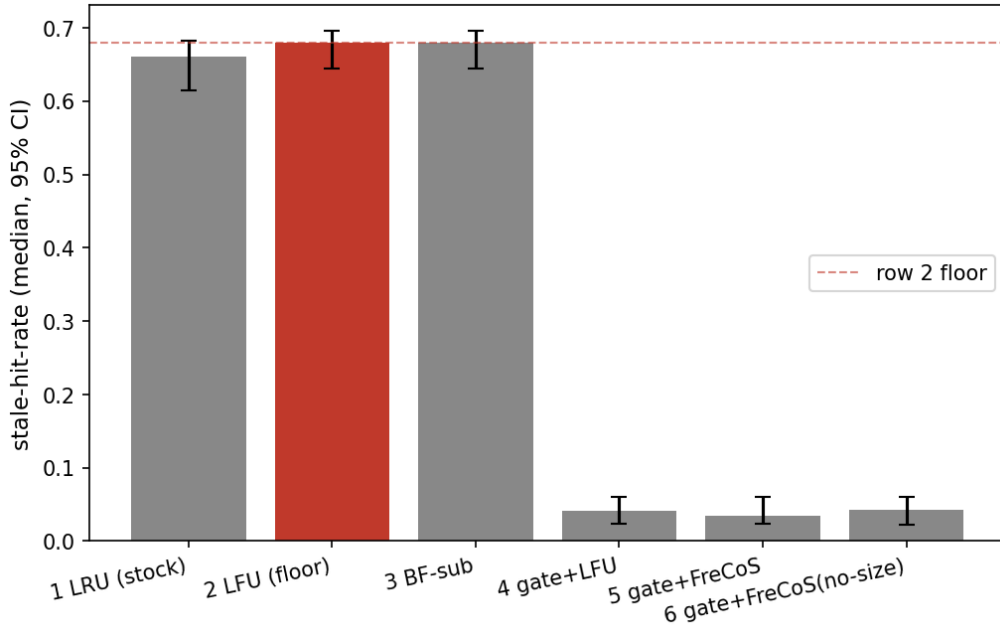


Figure 2: Stale-hit-rate across all six ablation rows, with 95 percent confidence intervals. Row two, LFU with the gate disabled, is marked as the floor of comparison.

0.017 (132 hits) at 30%, then flattens completely; the three largest points produce identical per-seed hit counts. The knee, where marginal hit-rate gain per additional entry drops by more than an order of magnitude relative to the previous step, falls at 30% of the working set: the slope drops roughly 13x into that point and is exactly zero afterward. This cache size anchors every other sweep here.

The next sweep produces the trade-off curve this report’s argument rests on most heavily: holding cache size fixed at that knee, I sweep the gate’s target confidence level.

Table 7: Raising TTL confidence from 0.80 to 0.99 buys a 16x reduction in stale-hit-rate for a 15% reduction in hit rate; most of the stale-hit-rate gain is captured in the first step, 0.80 to 0.90.

TTL confidence	Median stale-hit-rate	Median hit rate	Median n_hits
0.80	0.126	0.019	146
0.90	0.042	0.017	132
0.95	0.023	0.017	129
0.99	0.008	0.016	124

The exchange is not uniform: most of the stale-hit-rate benefit is captured in the first step, 0.80 to 0.90 (0.126 to 0.042), while hit rate erodes at a steadier pace across the full range. The cheapest correctness gains come early; pushing toward 0.99 buys a shrinking reduction in staleness for roughly the same ongoing hit-rate cost. This is a trade-off curve, not evidence a stricter gate is uniformly better: an operator chooses a point on it to suit their own tolerance for stale answers against misses.

A fourth sweep varied cluster count at 5, 10, 20, and 50. This came out non-monotone on both metrics, with every adjacent confidence interval overlapping heavily, so no directional effect is distinguishable from noise at 10 seeds per point. A plausible mechanism: more clusters shrinks each one’s calibration sample, making the learned rate noisier, while finer-grained clusters could in principle track heterogeneous half-lives more precisely, two effects pulling in opposite directions. This result sits in a supplementary table ([analysis/figures/supplementary.csv](#)) rather than a headline figure, since it clears neither the effect-size nor confidence-interval bar the rest of this report’s figures were held to.

6 Discussion

Fitting a staleness rate from observed data beats both alternatives I tested, a single pooled rate and no rate at all, by a large, significant margin, under an easier-than-fitted model, a matched model, and a genuinely adversarial mixture model (Section 5.1.1). Under the matched and easier models, learned tracks oracle almost exactly, narrower than the “strictly between the two brackets” shape I originally expected: it means per-cluster maximum-likelihood estimation is accurate on this workload, not that learning lands partway to a ceiling it approaches but cannot reach. Under the adversarial mixture, a real, non-degenerate gap between learned and oracle finally opens, though not one distinguishable from noise at $n = 10$. I report each finding as it actually came out rather than as I expected any of them to.

I expected the gate to carry most of the stale-hit-rate improvement with the decay term contributing less on top than alone. What I found is that the decay term’s contribution is not distinguishable from zero at all, at either cache size I tested. The gate is the correctness mechanism; the eviction term, as measured here, does not add detectable work on top of it. A system wanting only the staleness-correctness gain could plausibly adopt the gate alone. That gate is, at bottom, a time-to-live learned per cluster rather than set globally by hand; Section 1.1 dismisses plain time-to-live caching as what weaker products already do, and this project’s strongest result is a better time-to-live, not a mechanism outside that family. “Better”

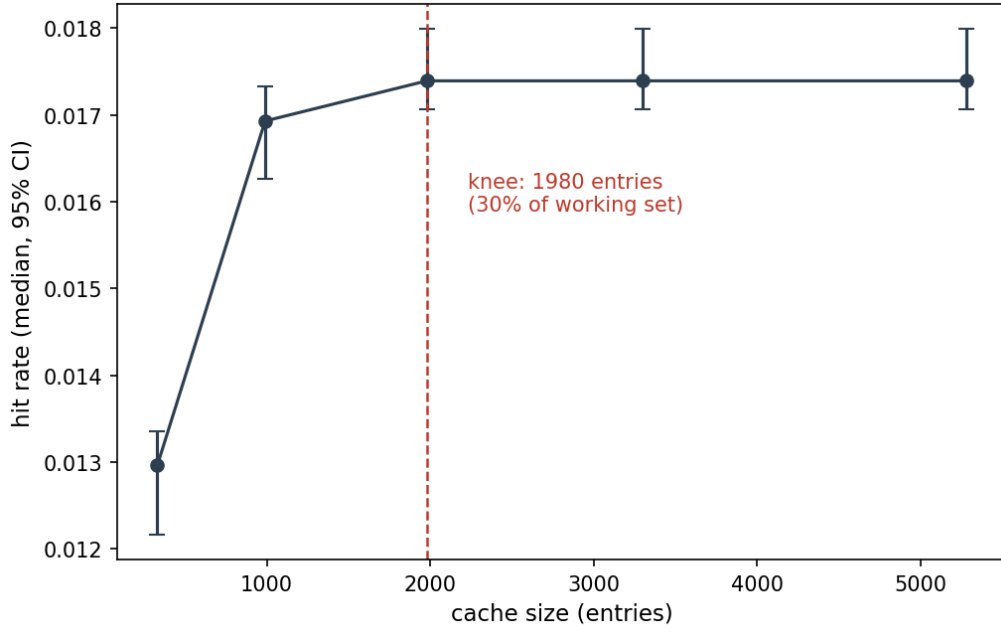


Figure 3: Hit rate against cache size, with the identified knee at thirty percent of the working set marked.

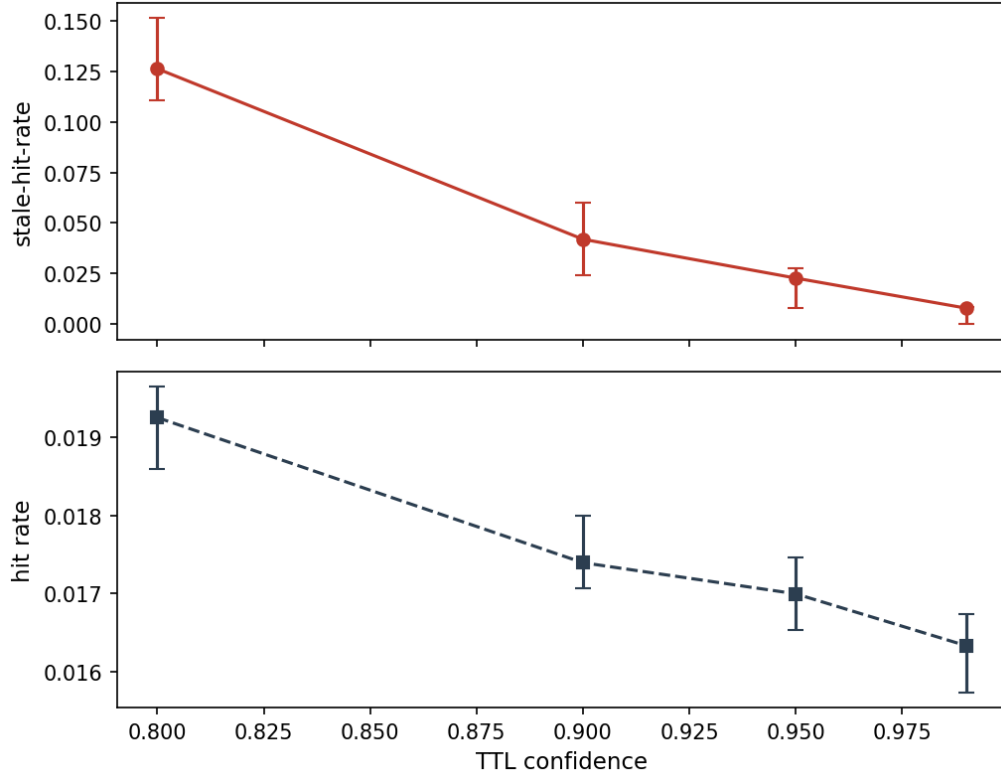


Figure 4: Stale-hit-rate and hit rate plotted together against the gate's target confidence level, showing the shape of the correctness-efficiency exchange rate rather than a single winning configuration.

is measured directly, not asserted: Section 5.1 is exactly the comparison between this learned per-cluster time-to-live and a single global one, learned’s median stale-hit-rate 0.035 against global’s 0.144, a significant 4x difference. The global arm there is the hand-set time-to-live a system without this project’s per-cluster fitting would use instead.

The correctness-efficiency trade-off in Section 5.3 most needs to be read as a trade-off, not a win: raising the gate’s target confidence buys a large stale-hit-rate reduction at a small, real hit-rate cost, front-loaded so early gains are cheap and later ones cost more. I do not claim a universally correct point on that curve.

Three limitations bound how far these results generalize. Every staleness result here comes from a synthetic, self-authored workload; I attempted external ground truth from Wikipedia and stopped after a feasibility spike measured 40% rule agreement against an 80% bar set in advance. The harness’s exact-match index caps every absolute rate well below a real semantic index and leaves false-hit-rate carrying no signal (Section 4.2); every finding is a relative comparison across conditions sharing that limitation, and I have not verified a semantic index would preserve the same rankings. Neither prior-art comparator, Biton and Friedman’s released policy nor Cortex’s LCFU, was actually run against this workload; the eviction axis is measured only against count-based policies.

7 Conclusion and future work

I set out to name a gap in how semantic caches are evaluated, stale-hit-rate, and to build a system that closes it. That system has two components, matching the two gaps named in Section 1.1: a staleness-aware serve-path gate, and a cost-aware eviction term. The first did the work I hoped for. The gate converts almost all stale hits into misses while removing only about 3% of useful hits, at under a microsecond of measured overhead per decision, and the learned per-cluster rate that feeds it beats naive pooling significantly and consistently across three tested half-life distributions, including a genuinely adversarial one. The second did not: the eviction value function’s cost and size terms behave correctly under every property test I wrote, but their measured contribution on top of an active gate is not distinguishable from zero at either cache size I tested. Of the two gaps this project set out to close, one is closed and measured; the other’s mechanism is implemented and tested but has not been shown to add value on this workload.

A real semantic index in place of the exact-match harness would likely change every absolute number here and is the change most likely to make results comparable to production; I expect the relative findings to survive it, but have not verified that. Running Biton and Friedman’s policy directly, rather than the documented substitute, would close the comparator gap in Section 6. Calibrating the generator’s distributions against a real inference trace and content-update corpus would strengthen external validity. Dynamic re-clustering under drift, a probabilistic staleness budget in the style of FreshCache, and admission control on top of the current policy remain future work, as in the original design.

Appendix A: correctness evidence

Correctness is checked at three levels. A reference oracle (`tests/oracle/reference_cache.py`) implements the same hit/miss/stale decision as the extension using a naive dict-and-linear-scan approach with no shared code, and every pipeline decision is cross-checked against it in `tests/test_stock_parity.py`. A callable invariant suite (`tests/invariants.py`) is run not only in unit tests but inside `benchmarks/harness.py` on every experimental run, checking budget bounds, argmin eviction selection, no entry served past its cluster’s time-to-live, and that staleness decisions derive from creation time, never last-access time. Property-based tests

in `tests/test_frecos.py` check the eviction value function’s monotonicity in each of its four terms independently.

The most persuasive of these artifacts is a property test that failed against my original design rather than a bug I found by inspection: `test_cold_start_entry_scores_above_zero` in `tests/test_frecos.py` asserts that a fresh entry must score above zero, and it failed against the literal $\log(1 + \text{freq})$ formula, which is how I found and fixed the $\log(1 + \text{freq}) \rightarrow \log(2 + \text{freq})$ issue described in Section 3.1.

Appendix B: code and data artifacts

This repository has not yet been pushed to a public remote; it is currently local-only. A `Dockerfile` and `environment.yml` build a working environment from a clean clone, and `.github/workflows/ci.yml` reruns the full test suite and a benchmark smoke test on every commit. `benchmarks/samples/smoke_row.csv` and `smoke_row.json` are committed sample benchmark logs reproducible with `make bench-smoke`.

Table 8: Where every artifact referenced in this report lives in the repository.

Artifact	Location
Pipeline seam (serve-path decision)	<code>gptcache_ext/pipeline.py</code>
Additive metadata storage	<code>gptcache_ext/metadata.py</code>
Staleness model (clustering, fitter, gate)	<code>gptcache_ext/staleness/</code>
Eviction (FreCoS and baselines)	<code>gptcache_ext/eviction/</code>
Synthetic workload generator (W1)	<code>workloads/w1_synthetic/</code>
Benchmark harness and metrics	<code>benchmarks/harness.py</code> , <code>metrics.py</code>
Experiment runners (Section 5)	<code>benchmarks/runners/</code>
Raw results per experiment	<code>results/brackets/</code> , <code>ablation/</code> , <code>sweeps/</code>
Figure regeneration (one command)	<code>cd analysis && python3 make_figures.py</code>
Reference oracle	<code>tests/oracle/reference_cache.py</code>
Invariant suite	<code>tests/invariants.py</code>
Property tests	<code>tests/test_frecos.py</code>
GPTCache pinned commit and source-map	<code>docs/baseline-source-map.md</code>
Wikipedia feasibility spike	<code>docs/w2-feasibility.md</code>
Docker / environment for a clean rebuild	<code>Dockerfile</code> , <code>environment.yml</code>
Continuous integration	<code>.github/workflows/ci.yml</code>
Sample benchmark logs	<code>benchmarks/samples/</code>

No figure in this report is hand-edited; every one is regenerated directly from a committed results CSV by the single command in Table 8.

References

- [1] N. Biton and R. Friedman. *From Exact Hits to Close Enough: Eviction Policies for Semantic Caches*. arXiv:2603.03301, 2026.
- [2] C. Ruan, C. Bi, K. Zheng, Z. Shi, X. Wan, and J. Li. *Cortex: Achieving Low-Latency, Cost-Efficient Remote Data Access for LLM via Semantic-Aware Knowledge Caching*. arXiv:2509.17360, 2026. NSDI 2026; earlier preprint circulated under the title “Asteria.”
- [3] L. Cherkasova and G. Ciardo. *Role of Aging, Frequency, and Size in Web Cache Replacement Policies*. Proceedings of the International Conference on High-Performance Computing and Networking (HPCN), 2001.

- [4] C. Wang, X. Liu, Y. Zhu, A. Youssef, P. Nagpurkar, and H. Chen. *Category-Aware Semantic Caching for Heterogeneous LLM Workloads*. arXiv:2510.26835, 2025.
- [5] M. Mansoor, T. Ahmad, and Y.-C. Yoon. *Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs*. arXiv:2607.04281, 2026.
- [6] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang, and J. Liu. *SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models*. arXiv:2406.00025, 2024.
- [7] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar. *MeanCache: User-Centric Semantic Caching for LLM Web Services*. arXiv:2403.02694, 2024.
- [8] G. Einziger and R. Friedman. *TinyLFU: A Highly Efficient Cache Admission Policy*. arXiv:1512.00727, 2015.
- [9] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez. *vCache: Verified Semantic Prompt Caching*. arXiv:2502.03771, 2025. ICLR 2026.
- [10] Zilliz. *GPTCache: A Library for Creating Semantic Cache for LLM Queries*. <https://github.com/zilliztech/GPTCache>, 2024. Version 0.1.44, commit bae7ffef774e762d9d4e60fce70be00011188a6.
- [11] P. Rajpurkar, R. Jia, and P. Liang. *Know What You Don't Know: Unanswerable Questions for SQuAD*. arXiv:1806.03822, 2018.